

Федеральное государственное автономное образовательное учреждение высшего образования «**Национальный исследовательский университет ИТМО**»

Факультет Программной Инженерии и Компьютерной Техники

Лабораторная работа №4
по дисциплине «**Основы программной инженерии**»

Вариант: **8483**

Преподаватель:
Лазеев Сергей Максимович

Выполнил: Давидюк Антон Юрьевич
Группа: P3210

Оглавление

1. Задание.....	3
2. Исходный код разработанных MBean	3
3. JConsole	6
4. VisualVM	10
5. Исследование программы на утечки памяти	14
6. Вывод.....	18

1. Задание

8483

Внимание! У разных вариантов разный текст задания!

1. Для своей программы из [лабораторной работы #3](#) по дисциплине "Веб-программирование" реализовать:

- MBean, считающий общее число установленных пользователем точек, а также число точек, попадающих в область. В случае, если пользователь совершил 3 "промаха" подряд, разработанный MBean должен отправлять оповещение об этом событии.
- MBean, определяющий средний интервал между кликами пользователя по координатной плоскости.

2. С помощью утилиты JConsole провести мониторинг программы:

- Снять показания MBean-классов, разработанных в ходе выполнения задания 1.
- Определить время (в мс), прошедшее с момента запуска виртуальной машины.

3. С помощью утилиты VisualVM провести мониторинг и профилирование программы:

- Снять график изменения показаний MBean-классов, разработанных в ходе выполнения задания 1, с течением времени.
- Определить имя класса, объекты которого занимают наибольший объём памяти JVM; определить пользовательский класс, в экземплярах которого находятся эти объекты.

4. С помощью утилиты VisualVM и профилировщика IDE NetBeans, Eclipse или Idea локализовать и устранить проблемы с производительностью в [программе](#). По результатам локализации и устранения проблемы необходимо составить отчёт, в котором должна содержаться следующая информация:

- Описание выявленной проблемы.
- Описание путей устранения выявленной проблемы.
- Подробное (со скриншотами) описание алгоритма действий, который позволил выявить и локализовать проблему.

Студент должен обеспечить возможность воспроизведения процесса поиска и локализации проблемы по требованию преподавателя.

2. Исходный код разработанных MBean

ClickInterval

```
package org.coordinate.monitor;
```

```
public class ClickInterval implements ClickIntervalMBean {
```

```

private long totalIntervalSum = 0;
private long totalIntervals = 0;

@Override
public synchronized double getAverageIntervalMs() {
    if (totalIntervals == 0) return 0.0;
    return (double) totalIntervalSum / totalIntervals;
}

@Override
public synchronized long getTotalIntervalsRecorded() {
    return totalIntervals;
}

@Override
public synchronized void registerInterval(long intervalMs) {
    if (intervalMs >= 0) {
        totalIntervalSum += intervalMs;
        totalIntervals++;
    }
}
}

```

ClickIntervalMBean

```

package org.coordinate.monitor;

public interface ClickIntervalMBean {
    double getAverageIntervalMs();
    long getTotalIntervalsRecorded();
    void registerInterval(long intervalMs);
}

```

JmxMonitorManager

```

package org.coordinate.monitor;

import jakarta.annotation.PostConstruct;
import jakarta.annotation.PreDestroy;
import jakarta.enterprise.context.ApplicationScoped;
import java.lang.management.ManagementFactory;
import javax.management.MBeanServer;
import javax.management.ObjectName;

@ApplicationScoped
public class JmxMonitorManager {
    private MBeanServer mBeanServer;
    private ObjectName statsName;
    private ObjectName intervalName;

    private PointStats pointStats;
    private ClickInterval clickInterval;

    private long lastClickTime = 0;

    @PostConstruct
    public void init() {
        mBeanServer = ManagementFactory.getPlatformMBeanServer();
        pointStats = new PointStats();
        clickInterval = new ClickInterval();

        try {
            statsName = new ObjectName("org.coordinate:type=PointStats");

```

```

        intervalName = new ObjectName("org.coordinate:type=ClickInterval");

        mBeanServer.registerMBean(pointStats, statsName);
        mBeanServer.registerMBean(clickInterval, intervalName);
    } catch (Exception e) {
        System.err.println("Ошибка регистрации MBean: " + e.getMessage());
    }
}

@PreDestroy
public void destroy() {
    try {
        if (statsName != null) mBeanServer.unregisterMBean(statsName);
        if (intervalName != null) mBeanServer.unregisterMBean(intervalName);
    } catch (Exception e) {
        System.err.println("Ошибка снятия MBean: " + e.getMessage());
    }
}

public synchronized void processClick(boolean isHit) {
    long now = System.currentTimeMillis();

    // 1. Расчёт и запись интервала
    if (lastClickTime > 0) {
        long interval = now - lastClickTime;
        clickInterval.registerInterval(interval);
    }
    lastClickTime = now;

    // 2. Регистрация статистики попаданий
    pointStats.registerPoint(isHit);
}
}

```

PointStats

```

package org.coordinate.monitor;

import javax.management.MBeanNotificationInfo;
import javax.management.Notification;
import javax.management.NotificationBroadcasterSupport;

public class PointStats extends NotificationBroadcasterSupport implements PointStatsMBean {
    private int totalPoints = 0;
    private int totalHits = 0;
    private int consecutiveMisses = 0;
    private int notificationSequence = 1;

    @Override
    public synchronized int getTotalPoints() {
        return totalPoints;
    }

    @Override
    public synchronized int getTotalHits() {
        return totalHits;
    }

    @Override
    public synchronized int getConsecutiveMisses() {
        return consecutiveMisses;
    }
}

```

```

@Override
public synchronized void registerPoint(boolean isHit) {
    totalPoints++;
    if (isHit) {
        totalHits++;
        consecutiveMisses = 0; // Сброс счётчика промахов при попадании
    } else {
        consecutiveMisses++;
        // Отправка оповещения при 3 промахах подряд
        if (consecutiveMisses == 3) {
            Notification notification = new Notification(
                "lab3.miss.alert",
                this,
                notificationSequence++,
                System.currentTimeMillis(),
                "Внимание: совершено 3 промаха подряд!"
            );
            sendNotification(notification);
        }
    }
}

@Override
public MBeanNotificationInfo[] getNotificationInfo() {
    String[] types = new String[]{"lab3.miss.alert"};
    String name = Notification.class.getName();
    String description = "Оповещение о серии из 3 промахов";
    return new MBeanNotificationInfo[]{new MBeanNotificationInfo(types, name, description)};
}
}

```

PointStatsMBean

```

package org.coordinate.monitor;

public interface PointStatsMBean {
    int getTotalPoints();
    int getTotalHits();
    int getConsecutiveMisses();
    void registerPoint(boolean isHit);
}

```

3. JConsole

Показания MBean

Overview Memory Threads Classes VM Summary MBeans

Tree view:

- JMImplementation
 - com.sun.management
 - java.lang
 - java.nio
 - java.util.logging
 - jboss.as
 - jboss.as.expr
 - jboss.jta
 - jboss.modules
 - jboss.msc
 - jboss.remoting.endpoint
 - jboss.remoting.handler
 - jboss.root
 - jboss.threads
 - jboss.ws
 - jdk.management.jfr
 - org.coordinate
 - ClickInterval
 - PointStats
 - Attributes
 - TotalPoints
 - TotalHits
 - ConsecutiveMisses
 - Operations
 - registerPoint
 - Notifications
 - org.wildfly.clustering.infinispan
 - org.xnio

MBean details:

Attribute value

| Name | Value |
|-------------------|-------|
| ConsecutiveMisses | 4 |

Refresh

MBeanAttributeInfo

| Name | Value |
|-------------|----------------------------------|
| Attribute: | |
| Name | ConsecutiveMisses |
| Description | Attribute exposed for management |
| Readable | true |
| Writable | false |
| Is | false |
| Type | int |

Descriptor

| Name | Value |
|------|-------|
|------|-------|

Tree view:

- JMImplementation
 - com.sun.management
 - java.lang
 - java.nio
 - java.util.logging
 - jboss.as
 - jboss.as.expr
 - jboss.jta
 - jboss.modules
 - jboss.msc
 - jboss.remoting.endpoint
 - jboss.remoting.handler
 - jboss.root
 - jboss.threads
 - jboss.ws
 - jdk.management.jfr
 - org.coordinate
 - ClickInterval
 - Attributes
 - AverageIntervalMs
 - TotalIntervalsRecorded
 - Operations
 - registerInterval
 - PointStats
 - org.wildfly.clustering.infinispan
 - org.xnio

MBean details:

Attribute value

| Name | Value |
|-------------------|--------------------|
| AverageIntervalMs | 242.53846153846155 |

Refresh

MBeanAttributeInfo

| Name | Value |
|-------------|----------------------------------|
| Attribute: | |
| Name | AverageIntervalMs |
| Description | Attribute exposed for management |
| Readable | true |
| Writable | false |
| Is | false |
| Type | double |

Descriptor

| Name | Value |
|------|-------|
|------|-------|

Время работы

понедельник, 22 июня 2026 г., 13:41:49 Москва, стандартное время

Uptime: 7 minutes
Process CPU time: 1 minute
JIT compiler: HotSpot 64-Bit Tiered Compilers
Total compile time: 58,260 seconds

Total compile time: 58,260 seconds

Name: 25468@Krijanovski

Current classes loaded: 24 598

Total classes loaded: 24 598

Total classes unloaded: 0

Total threads started: 218

Committed memory: 203 776 kbytes

Pending finalization: 0 objects

Garbage collector: Name = 'G1 Old Generation', Collections = 0, Total time spent = 0.000 seconds

Total physical memory: 16 641 956 kbytes

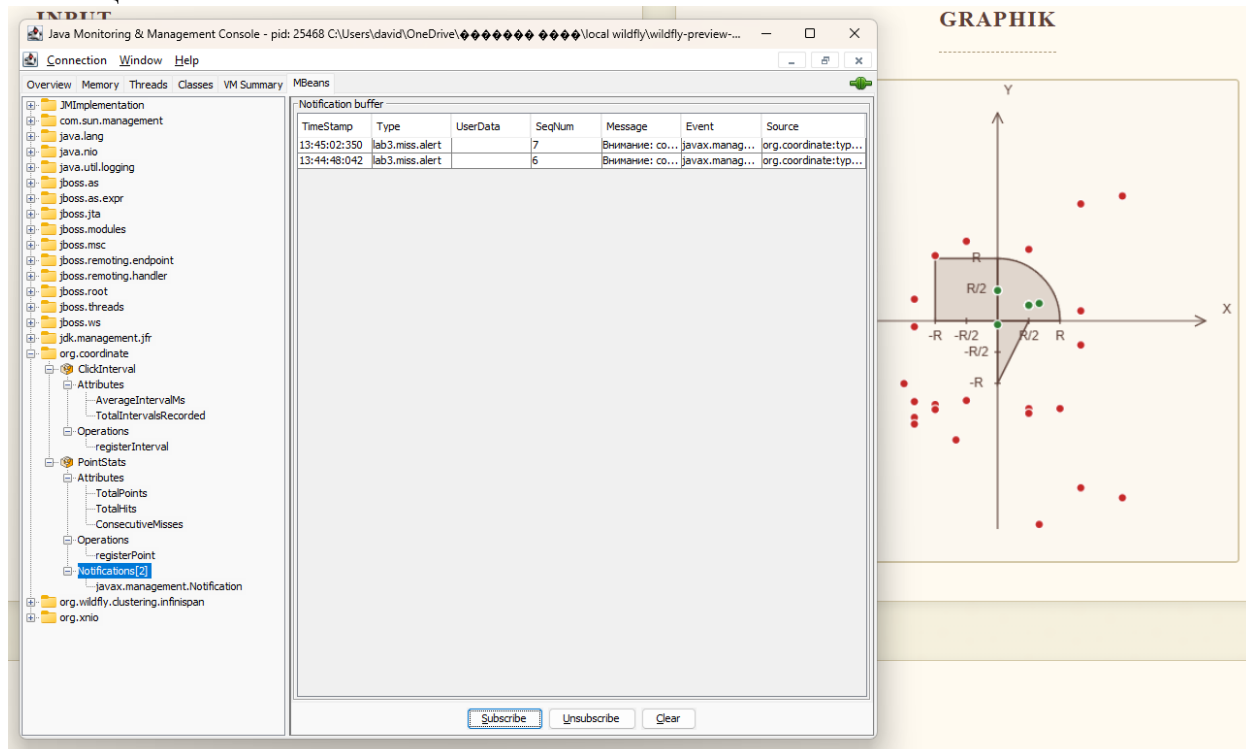
Free physical memory: 2 471 536 kbytes

Total swap space: 33 283 912 kbytes

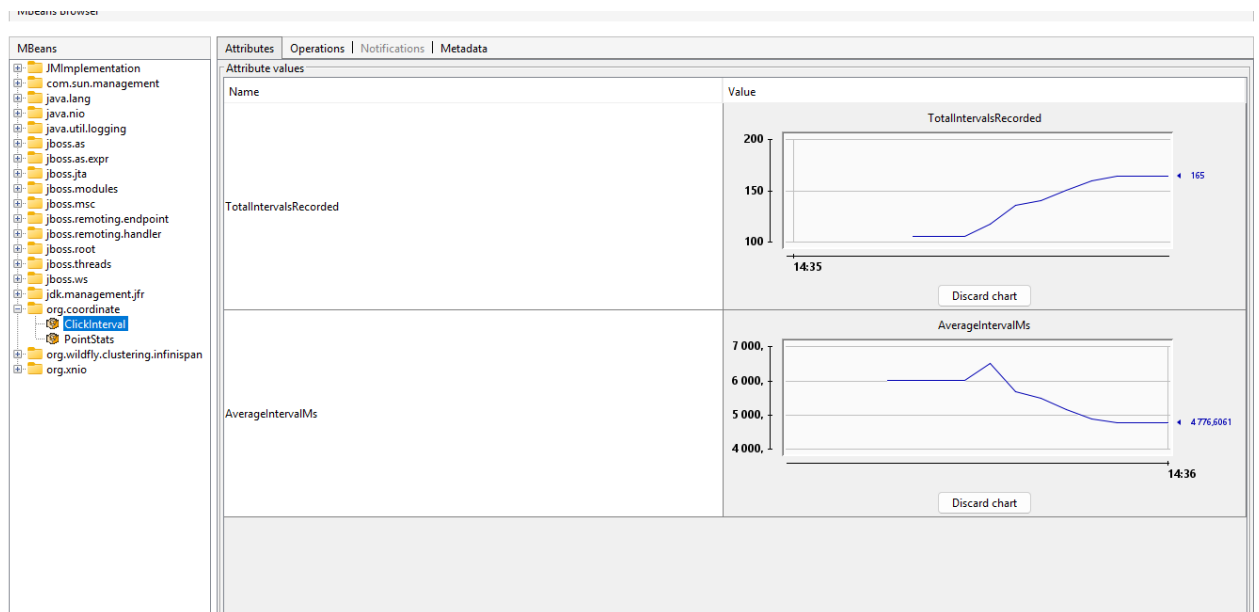
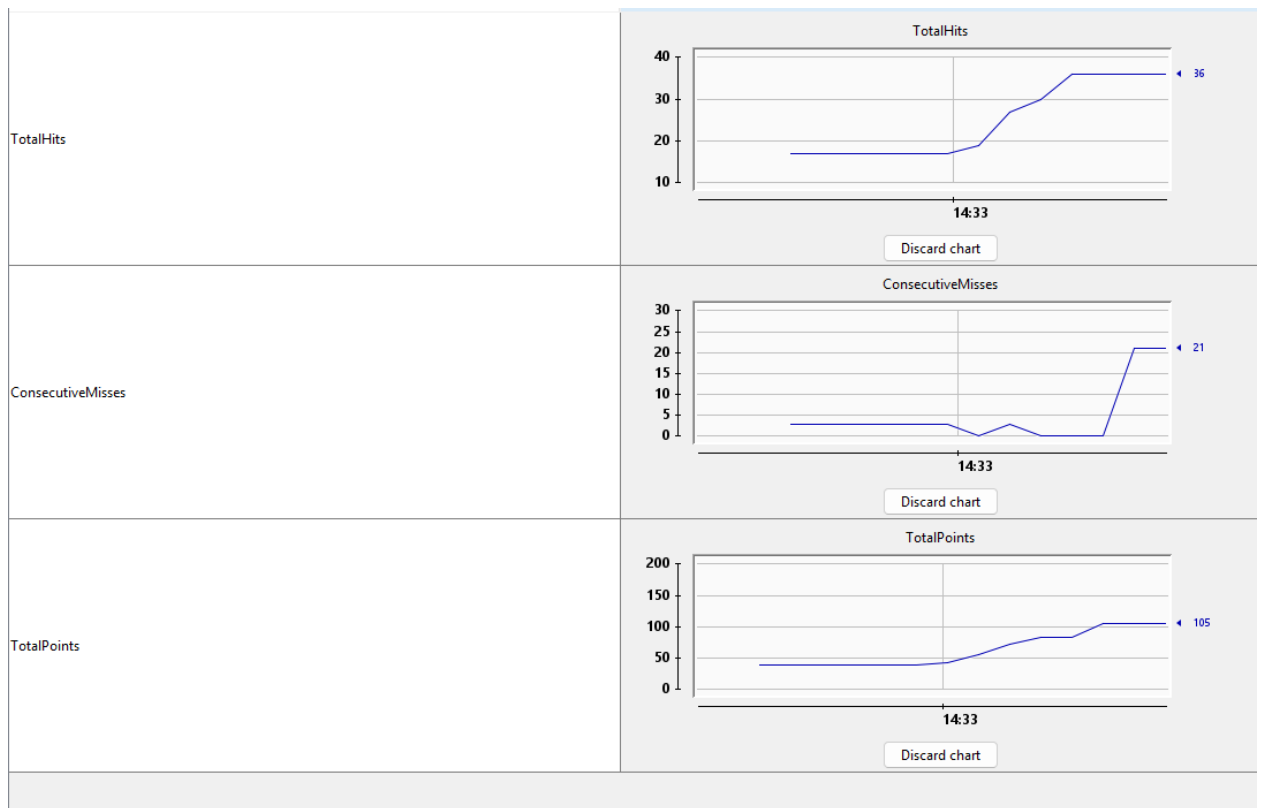
Free swap space: 8 701 932 kbytes

VM arguments: -Dprogram.name=standalone.bat -Xms64M -Xmx512M -XX:MetaspaceSize=96M -XX:MaxMetaspaceSize=256m -Djava.net.preferIPv4Stack=true -Djboss.modules.system.pkgs=org.jboss.byteman -Djava.awt.headless=true --add-exports=java.desktop/sun.awt=ALL-UNNAMED --add-exports=java.naming/com.sun.jndi ldap=ALL-UNNAMED --add-exports=java.naming/com.sun.jndi.url.ldap=ALL-UNNAMED --add-exports=java.naming/com.sun.jndi.url.ldap.all=ALL-UNNAMED --add-exports=jdk.management/com.sun.management=ALL-UNNAMED

Оповещения



Графики



JVM Overview

Start Page x jboss-modules.jar (pid 10424) x

OverviewMonitorThreadsSamplerProfilerMBeans

jboss-modules.jar (pid 10424)

Overview

PID: 10424

Host: localhost

Main class: jboss-modules.jar

Arguments: -mp C:\Users\david\OneDrive\local wildfly\wildfly-preview-26.1.3.Final\modules org.jboss.as.standalone -Djboss.home.dir=C:\Users\david\OneDrive\local wildfly\wildfly-preview-26.1.3.Final

JVM: Java HotSpot(TM) 64-Bit Server VM (17.0.12+8-LTS-286, mixed mode, sharing)

Java version: 17.0.12 2024-07-16 LTS, vendor: Oracle Corporation

Java Home: C:\Program Files\Java\jdk-17

JVM Flags: <none>

Heap dump on OOME: disabled

Saved data x

JVM argumentsSystem properties

Thread Dumps: 0

Heap Dumps: 0

Profiler Snapshots: 0

JFR Snapshots: 0

-Dprogram.name=standalone.bat

-Xms64M

-Xmx512M

-XX:MetaspaceSize=96M

-XX:MaxMetaspaceSize=256m

-Djava.net.preferIPv4Stack=true

-Djboss.modules.system.pkgs=org.jboss.byteman

-Djava.awt.headless=true

--add-exports=java.desktop/sun.awt=ALL-UNNAMED

--add-exports=java.naming/com.sun.jndi.idap=ALL-UNNAMED

--add-exports=java.naming/com.sun.jndi.url.idap=ALL-UNNAMED

--add-opens=java.base/java.lang=ALL-UNNAMED

--add-opens=java.base/java.lang.invoke=ALL-UNNAMED

--add-opens=java.base/java.io=ALL-UNNAMED

--add-opens=java.base/java.lang.reflect=ALL-UNNAMED

--add-opens=java.base/java.security=ALL-UNNAMED

--add-opens=java.base/java.util=ALL-UNNAMED

--add-opens=java.base/java.util.concurrent=ALL-UNNAMED

--add-opens=java.management/javax.management=ALL-UNNAMED

--add-opens=java.naming/javax.naming=ALL-UNNAMED

-Djava.security.manager=allow

-Dorg.jboss.boot.log.file=C:\Users\david\OneDrive\local wildfly\wildfly-preview-26.1.3.Final\standalone\log\server.log

-Dlogging.configuration=file:C:\Users\david\OneDrive\local wildfly\wildfly-preview-26.1.3.Final\standalone\configuration\logging.properties

Мониторинг:

jboss-modules.jar (pid 19568) x

OverviewMonitorThreadsSamplerProfilerMBeans[heapdump] 14:37:55 x

jboss-modules.jar (pid 19568)

Monitor

CPU

Memory

Classes

Threads

Uptime: 17 min 39 sec

Perform GCHeap Dump

CPU

HeapMetaspace

Classes

Threads

CPU usage: 0,1 %

GC activity: 0,0 %

Size: 269 484 064 B

Max: 536 870 936 B

Used: 137 402 736 B

Total loaded: 24 950

Total unloaded: 126

Shared loaded: 1 237

Shared unloaded: 0

Live: 94

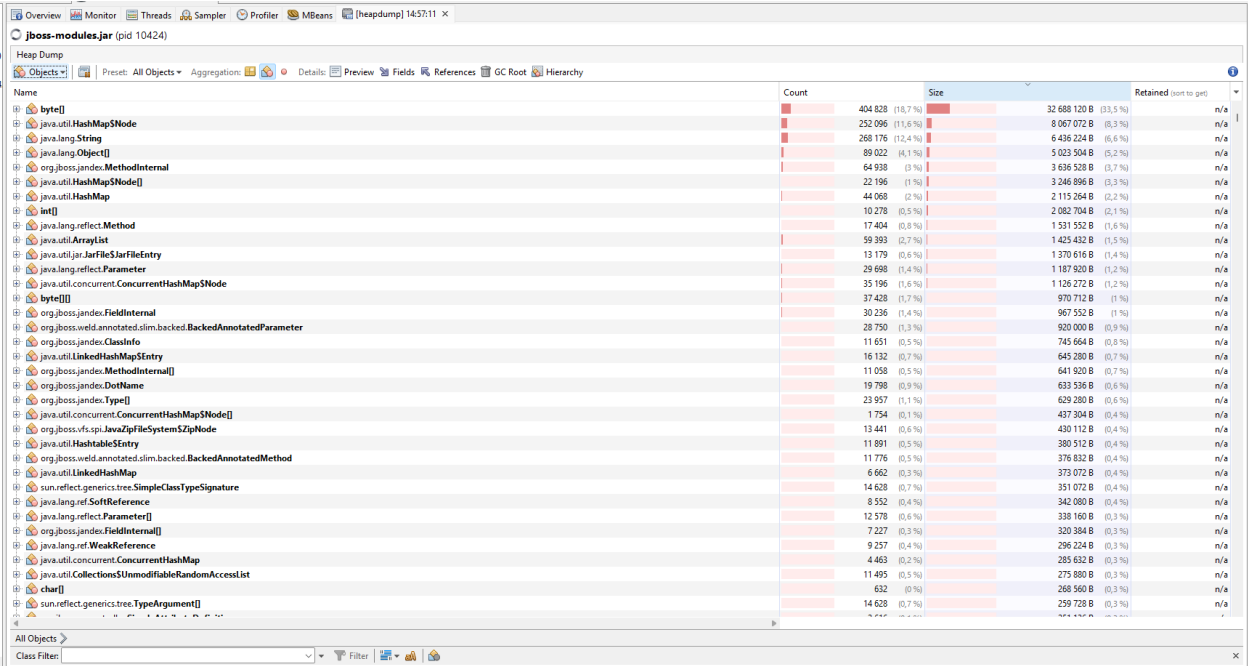
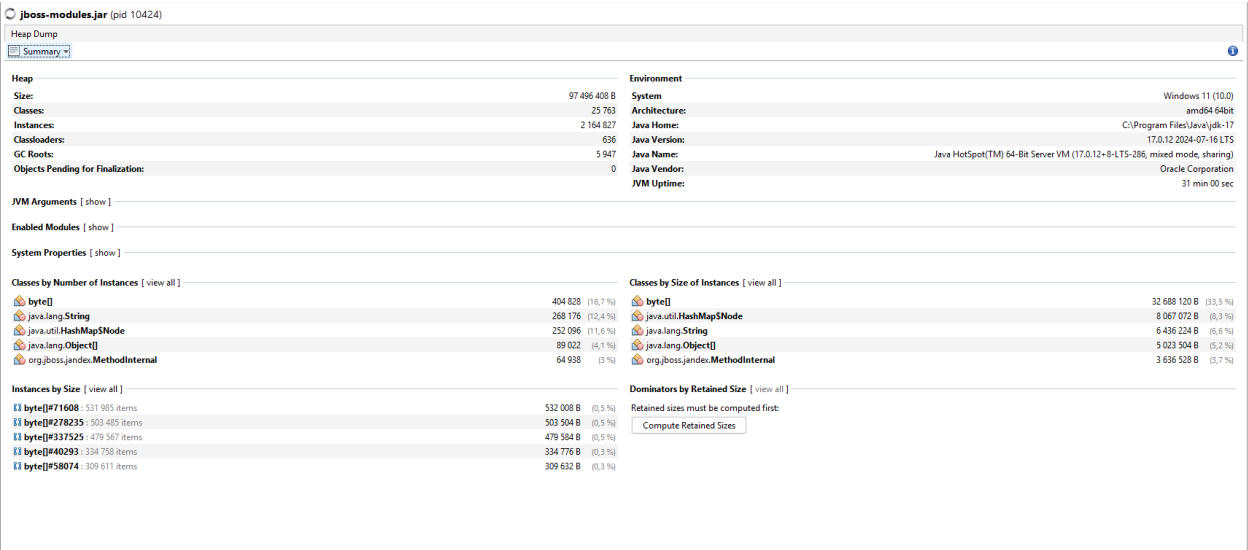
Live peak: 209

Daemon: 31

Total started: 230

The dashboard displays four main monitoring panels for jboss-modules.jar (pid 19568). The CPU panel shows usage at 0.1% and GC activity at 0.0%. The Memory panel shows heap size at 269,484,064 B, max at 536,870,936 B, and used at 137,402,736 B. The Classes panel shows 24,950 total loaded classes and 126 total unloaded classes. The Threads panel shows 94 live threads, a live peak of 209, 31 daemon threads, and a total of 230 started threads. All metrics are plotted over a time range from 14:28 to 14:38.

Определить имя класса, объекты которого занимают наибольший объём памяти JVM; определить пользовательский класс, в экземплярах которого находятся эти объекты.



| Start Page x jboss-modules.jar (pid 10424) x | | | | |
|---|-----------------|----------------------|------------------------|--|
| Overview Monitor Threads Sampler Profiler MBeans [heapdump] 14:57:11 x | | | | |
| jboss-modules.jar (pid 10424) | | | | |
| Heap Dump | | | | |
| Objects ▾ Preset: All Objects ▾ Aggregation: ▾ Details: ▾ Preview ▾ Fields ▾ References ▾ GC Root ▾ Hierarchy | | | | |
| Name | Count | Size | Retained (sort to get) | |
| byte[] | 404 828 (16,7%) | 32 688 120 B (33,5%) | 31 235 888 B (32,1%) | |
| java.lang.String | 268 176 (12,4%) | 6 436 224 B (6,6%) | 19 892 648 B (20,4%) | |
| java.util.HashMap\$Node | 252 096 (11,6%) | 8 067 072 B (8,3%) | 36 597 152 B (37,5%) | |
| java.lang.Object[] | 89 022 (4,1%) | 5 023 504 B (5,2%) | 22 237 984 B (22,8%) | |
| org.jboss.jandex.MethodInternal | 64 938 (3%) | 3 636 528 B (3,7%) | 7 984 952 B (8,2%) | |
| java.util.ArrayList | 59 393 (2,7%) | 1 425 432 B (1,5%) | 12 941 288 B (13,3%) | |
| java.util.HashMap | 44 068 (2%) | 2 115 264 B (2,2%) | 40 313 224 B (41,3%) | |
| byte[] | 37 428 (1,7%) | 970 712 B (1%) | 2 921 864 B (3%) | |
| java.util.concurrent.ConcurrentHashMap\$Node | 33 196 (1,6%) | 1 126 272 B (1,2%) | 5 398 616 B (5,5%) | |
| org.jboss.jandex.FieldInternal | 30 236 (1,4%) | 967 552 B (1%) | 1 746 456 B (1,8%) | |
| java.lang.reflect.Parameter | 29 698 (1,4%) | 1 187 920 B (1,2%) | 2 614 000 B (2,7%) | |
| org.jboss.weld.annotated.slim.backed.BackedAnnotatedParameter | 28 750 (1,3%) | 920 000 B (0,9%) | 920 024 B (0,9%) | |
| org.jboss.jandex.Type[] | 23 957 (1,1%) | 629 280 B (0,6%) | 714 104 B (0,7%) | |
| java.util.HashMap\$Node[] | 22 196 (1%) | 3 246 896 B (3,3%) | 39 323 840 B (40,3%) | |
| org.jboss.jandex.DotName | 19 798 (0,9%) | 633 536 B (0,6%) | 1 471 440 B (1,5%) | |
| java.lang.reflect.Method | 17 404 (0,8%) | 1 531 552 B (1,6%) | 4 932 184 B (5,1%) | |
| java.util.LinkedHashMap\$Entry | 16 132 (0,7%) | 645 280 B (0,7%) | 2 122 432 B (2,2%) | |
| sun.reflect.generics.tree.SimpleClassTypeSignature | 14 628 (0,7%) | 351 072 B (0,4%) | 1 754 344 B (1,8%) | |
| sun.reflect.generics.tree.TypeArgument[] | 14 628 (0,7%) | 259 728 B (0,3%) | 835 440 B (0,9%) | |
| sun.reflect.generics.tree.ClassTypeSignature | 14 550 (0,7%) | 232 800 B (0,2%) | 2 529 704 B (2,6%) | |
| org.jboss.vfs.spi.javazip.FileSystemZipNode | 13 441 (0,6%) | 430 112 B (0,4%) | 4 640 136 B (4,8%) | |
| java.util.jar.JarFile\$JarFileEntry | 13 179 (0,6%) | 1 370 616 B (1,4%) | 2 731 680 B (2,8%) | |
| java.lang.reflect.Parameter[] | 12 578 (0,6%) | 338 160 B (0,3%) | 420 528 B (0,4%) | |
| java.util.HashMap\$Entry | 11 891 (0,5%) | 380 512 B (0,4%) | 1 169 224 B (1,2%) | |
| org.jboss.weld.annotated.slim.backed.BackedAnnotatedMethod | 11 776 (0,5%) | 376 632 B (0,4%) | 8 226 040 B (8,4%) | |
| org.jboss.jandex.ClassInfo | 11 651 (0,5%) | 745 664 B (0,8%) | 11 507 792 B (11,8%) | |
| java.util.Collections\$UnmodifiableRandomAccessList | 11 495 (0,5%) | 275 880 B (0,3%) | 1 973 640 B (2%) | |
| org.jboss.jandex.MethodInternal[] | 11 058 (0,5%) | 641 920 B (0,7%) | 7 176 656 B (7,4%) | |
| org.jboss.weld.util.collections.ImmutableArrayList | 10 329 (0,5%) | 247 088 B (0,3%) | 1 335 592 B (1,4%) | |
| int[] | 10 278 (0,5%) | 2 082 704 B (2,1%) | 2 082 208 B (2,1%) | |
| java.lang.ref.WeakReference | 9 257 (0,4%) | 296 224 B (0,3%) | 269 632 B (0,3%) | |
| java.lang.ref.SoftReference | 8 552 (0,4%) | 342 080 B (0,4%) | 178 480 B (0,2%) | |
| java.util.concurrent.ConcurrentSkipListMap\$Node | 8 148 (0,4%) | 195 552 B (0,2%) | 977 760 B (1%) | |
| java.lang.String[] | 8 048 (0,4%) | 226 232 B (0,2%) | 325 904 B (0,3%) | |
| org.jboss.jandex.ClassType | 7 982 (0,4%) | 191 568 B (0,2%) | 210 152 B (0,2%) | |

| | | | | |
|---|----------|------------------------|--------------------|--|
| Name | | | | |
| Count | Size | Retained (sort to get) | | |
| org.jboss.modules.ModuleClassLoader\$1 | 386 (0%) | 6 176 B (0%) | | |
| org.jboss.modules.Module | 386 (0%) | 21 616 B (0%) | 6 234 064 B (6,4%) | |
| org.jboss.modules.ModuleClassLoader | 386 (0%) | 37 056 B (0%) | 5 055 656 B (5,2%) | |
| org.jboss.as.controller.descriptions.DefaultResourceAddDescriptionProvider | 385 (0%) | 9 240 B (0%) | 11 200 B (0%) | |
| org.jboss.as.controller.capability.registry.CapabilityRegistration | 382 (0%) | 9 168 B (0%) | 95 528 B (0,1%) | |
| javax.management.ImmutableDescriptor | 378 (0%) | 9 072 B (0%) | 34 256 B (0%) | |
| java.util.Collections\$SynchronizedRandomAccessList | 373 (0%) | 8 952 B (0%) | 20 736 B (0%) | |
| org.jboss.weld.annotated.enhanced.jfr.ConstructorSignatureImpl | 369 (0%) | 5 904 B (0%) | 11 840 B (0%) | |
| org.jboss.as.controller.registry.BackResource | 368 (0%) | 11 776 B (0%) | 305 144 B (0,3%) | |
| jdk.jfr.internal.EventControl\$NamedControl | 366 (0%) | 8 784 B (0%) | 69 720 B (0,1%) | |
| jdk.jfr.internal.Control | 366 (0%) | 14 640 B (0%) | 60 936 B (0,1%) | |
| jdk.jfr.SettingDescriptor | 366 (0%) | 11 712 B (0%) | 20 496 B (0%) | |
| java.util.HashSet | 366 (0%) | 14 640 B (0%) | 14 640 B (0%) | |
| org.jboss.modules.ModuleClassLoaderDependency | 361 (0%) | 14 440 B (0%) | 14 440 B (0%) | |
| org.jboss.modules.ModuleClassLoaderDependencySpec | 361 (0%) | 14 440 B (0%) | 14 440 B (0%) | |
| org.jboss.weld.util.collections.ImmutableTinySet\$Singleton | 359 (0%) | 5 744 B (0%) | 10 776 B (0%) | |
| sun.security.x509.AlgorithmId | 354 (0%) | 8 496 B (0%) | 41 208 B (0%) | |
| java.util.WeakHashMap\$Entry | 352 (0%) | 14 080 B (0%) | 716 992 B (0,7%) | |
| org.jboss.modules.ResourceLoaderSpec | 349 (0%) | 9 312 B (0%) | 165 976 B (0,2%) | |
| org.jboss.modules.MutativeLibraryResourceLoader | 347 (0%) | 5 552 B (0%) | 130 504 B (0,1%) | |
| org.jboss.as.server.deployment.RegisteredDeploymentUnitProcessor | 344 (0%) | 8 256 B (0%) | 26 664 B (0%) | |
| org.jboss.weld.annotated.slim.backed.BackedAnnotatedConstructor | 343 (0%) | 10 976 B (0%) | 25 040 B (0%) | |
| org.eclipse.persistence.internal.helper.linkedlist.LinkedNode | 342 (0%) | 8 208 B (0%) | 12 208 B (0%) | |
| jdk.internal.math.FDBigInteger | 341 (0%) | 10 912 B (0%) | 35 712 B (0%) | |
| org.eclipse.persistence.internal.identitymaps.HardCacheWeakIdentityMap\$ReferenceCacheKey | 340 (0%) | 48 960 B (0,1%) | 125 920 B (0,1%) | |
| org.coordinate.beam.ResultBean\$ResultDTO | 340 (0%) | 10 880 B (0%) | 49 920 B (0,1%) | |
| sun.security.ssl.CipherSuite | 338 (0%) | 18 928 B (0%) | 27 952 B (0%) | |
| org.jboss.modules.filter.PathFilter | 334 (0%) | 9 416 B (0%) | 39 416 B (0%) | |
| org.jboss.modules.filter.MultiplePathFilter | 333 (0%) | 10 656 B (0%) | 58 040 B (0,1%) | |
| java.lang.invoke.LambdaForm\$NamedFunction | 332 (0%) | 7 968 B (0%) | 17 976 B (0%) | |
| java.lang.invoke.MethodTypeForm | 327 (0%) | 10 464 B (0%) | 61 136 B (0,1%) | |

Выводы по результатам мониторинга и профилирования:

Наибольший объём памяти занимают объекты класса byte[].

В момент снятия дампа памяти было обнаружено 404 828 экземпляров данного класса, которые суммарно занимали 32 688 120 байт (33,5% памяти JVM).

Второе место занимают объекты класса java.util.HashMap\$Node 8 067 072 байта (8,3%), третье место объекты класса java.lang.String 6 436 224 байта (6,6%).

Анализ ссылок (References) показал, что большая часть объектов byte[] связана с инфраструктурными компонентами сервера приложений WildFly.

Это свидетельствует о том, что значительная часть памяти используется для хранения содержимого JAR-архивов, индексов классов и метаданных, загружаемых сервером приложений при работе приложения.

ResultBean.ResultDTO, используемые для хранения результатов проверки координат и отображения истории запросов. Однако их суммарное потребление памяти незначительно по сравнению с инфраструктурными компонентами WildFly и не оказывает существенного влияния на общее потребление памяти JVM

5. Исследование программы на утечки памяти

```
44 WebRequest request = new GetMethodWebRequest( urlString: "http://test.meterw
45 while (true) {
46     WebResponse response = sc.getResponse(request);
47     System.out.println("Count: " + number++ + response);
48     java.lang.Thread.sleep( millis: 200);
```

В ходе анализа производительности приложения была выявлена проблема, связанная с частыми вызовами метода `java.lang.Thread.sleep(200)`. Этот метод приостанавливает выполнение потока на 200 миллисекунд, что влечёт за собой необоснованные задержки и снижает общую эффективность работы системы.

Поскольку данный вызов не выполняет полезных действий и не влияет на логику работы приложения, его можно безопасно удалить. Это позволит устранить лишние паузы и повысить производительность.

Зададим максимальный размер кучи равным 30 МБ с помощью параметра `-Xmx30m`, указав его при запуске программы через командную строку.

```
[java] Count: 66236[ _response = com.meterware.servletunit.ServletUnitHttpRespon
[java] Count: 66237[ _response = com.meterware.servletunit.ServletUnitHttpRespon
[java] Exception in thread "main" java.lang.OutOfMemoryError: Java heap space
[java]   at java.base/java.util.Arrays.copyOf(Arrays.java:3745)
[java]   at java.base/java.lang.AbstractStringBuilder.ensureCapacityInternal(AbstractStringBuilder.java:172)
```

С помощью `Heap Dump` найдем объекты, занимающие большую часть памяти.

Applications x Start Page x Main (pid 12852) x

Overview Monitor Threads Sampler Profiler [heapdump] 07:25:36 x

Main (pid 12852)

Heap Dump

Objects v Preset: All Objects Aggregation: Details Preview Fields References GC Root Hierarchy

| Name | Count | Size | Retained (sort to get) |
|--------------------------------|-----------------|----------------------|------------------------|
| byte[] | 28 785 (19.9 %) | 3 240 768 B (38.3 %) | n/a |
| java.lang.String | 27 622 (19.1 %) | 662 928 B (7.8 %) | n/a |
| java.lang.String#1 : ug-Cyrl | | 24 B (0 %) | n/a |
| java.lang.String#2 : tg-Arab | | 24 B (0 %) | n/a |
| java.lang.String#3 : sw-Arab | | 24 B (0 %) | n/a |
| java.lang.String#4 : so-Arab | | 24 B (0 %) | n/a |
| java.lang.String#5 : shi-Latn | | 24 B (0 %) | n/a |
| java.lang.String#6 : sd-Sind | | 24 B (0 %) | n/a |
| java.lang.String#7 : sd-Khoj | | 24 B (0 %) | n/a |
| java.lang.String#8 : sd-Deva | | 24 B (0 %) | n/a |
| java.lang.String#9 : sat-Deva | | 24 B (0 %) | n/a |
| java.lang.String#10 : pa-Arab | | 24 B (0 %) | n/a |
| java.lang.String#11 : ms-Arab | | 24 B (0 %) | n/a |
| java.lang.String#12 : mni-Mtei | | 24 B (0 %) | n/a |
| java.lang.String#13 : mn-Mong | | 24 B (0 %) | n/a |
| java.lang.String#14 : ml-Arab | | 24 B (0 %) | n/a |
| java.lang.String#15 : ky-Latn | | 24 B (0 %) | n/a |
| java.lang.String#16 : ky-Arab | | 24 B (0 %) | n/a |
| java.lang.String#17 : ku-Arab | | 24 B (0 %) | n/a |
| java.lang.String#18 : ks-Deva | | 24 B (0 %) | n/a |
| java.lang.String#19 : kk-Arab | | 24 B (0 %) | n/a |
| java.lang.String#20 : ki-Latn | | 24 B (0 %) | n/a |
| java.lang.String#21 : ha-Arab | | 24 B (0 %) | n/a |
| java.lang.String#22 : ff-Arab | | 24 B (0 %) | n/a |
| java.lang.String#23 : ff-Adlm | | 24 B (0 %) | n/a |
| java.lang.String#24 : en-Shaw | | 24 B (0 %) | n/a |
| java.lang.String#25 : dyo-Arab | | 24 B (0 %) | n/a |
| java.lang.String#26 : bmn-Nkoo | | 24 B (0 %) | n/a |
| java.lang.String#27 : az-Cyrl | | 24 B (0 %) | n/a |
| java.lang.String#28 : en-IO1 | | 24 B (0 %) | n/a |

All Objects String Filter

| Name | Count | Size | Retained (sort to get) |
|--|-----------------|----------------------|------------------------|
| byte[] | 28 785 (19.9 %) | 3 240 768 B (38.3 %) | n/a |
| java.lang.String | 27 622 (19.1 %) | 662 928 B (7.8 %) | n/a |
| java.util.concurrent.ConcurrentHashMap\$Node | 5 849 (4 %) | 187 168 B (2.2 %) | n/a |
| java.lang.ref.WeakReference | 5 397 (3.7 %) | 172 704 B (2 %) | n/a |
| java.lang.Class[] | 5 054 (3.5 %) | 115 080 B (1.4 %) | n/a |
| java.lang.Object[] | 5 039 (3.5 %) | 287 952 B (3.4 %) | n/a |
| java.util.HashMap\$Node | 4 584 (3.2 %) | 146 688 B (1.7 %) | n/a |
| java.util.LinkedHashMap\$Entry | 3 853 (2.7 %) | 154 120 B (1.8 %) | n/a |
| java.lang.ref.SoftReference | 2 843 (2 %) | 113 720 B (1.3 %) | n/a |
| java.lang.reflect.Method | 2 763 (1.9 %) | 243 144 B (2.9 %) | n/a |
| java.lang.Class\$ReflectionData | 2 596 (1.8 %) | 166 144 B (2 %) | n/a |

Исследовав кучу, находим повторяющиеся строки:

java.lang.Object[]#3431 : 106 710 items

<items>

- [0] = java.lang.String#4633 : Script 'document.wr("Hello Document")' failed: TypeError: undefined is not a function. (httpunit;)
- [1] = java.lang.String#4632 : Script 'document.wr("Hello Document")' failed: TypeError: undefined is not a function. (httpunit;)
- [2] = java.lang.String#4631 : Script 'document.wr("Hello Document")' failed: TypeError: undefined is not a function. (httpunit;)
- [3] = java.lang.String#4630 : Script 'document.wr("Hello Document")' failed: TypeError: undefined is not a function. (httpunit;)
- [4] = java.lang.String#4629 : Script 'document.wr("Hello Document")' failed: TypeError: undefined is not a function. (httpunit;)
- [5] = java.lang.String#4628 : Script 'document.wr("Hello Document")' failed: TypeError: undefined is not a function. (httpunit;)

java.lang.Object[]#3431 : 106 710 items

<items>

<references>

- elementData in java.util.ArrayList#24 : 82 384 elements
- static _errorMessages in class com.meterware.httpunit.javascript.JavaScript : JavaScript

Объекты `errorMessage` хранятся в `ArrayList`

```
40     public class JavaScript {
46         private static ArrayList _errorMessagees = new ArrayList();
47
48     }
```

Найдем строчку с добавлением объектов в этот список:

```
        throw new ScriptException( errorMessage );
    } else {
        _errorMessagees.add( errorMessage );
    }
```

В результате получается накопление `_errorMessagees` в списке, за счет чего и получается переполнение памяти. В программе есть функция для очистки `_errorMessage`, однако мы можем увидеть, что на самом деле она не используется

```
static void clearErrorMessages() {
    _errorMessagees.clear();
}
```

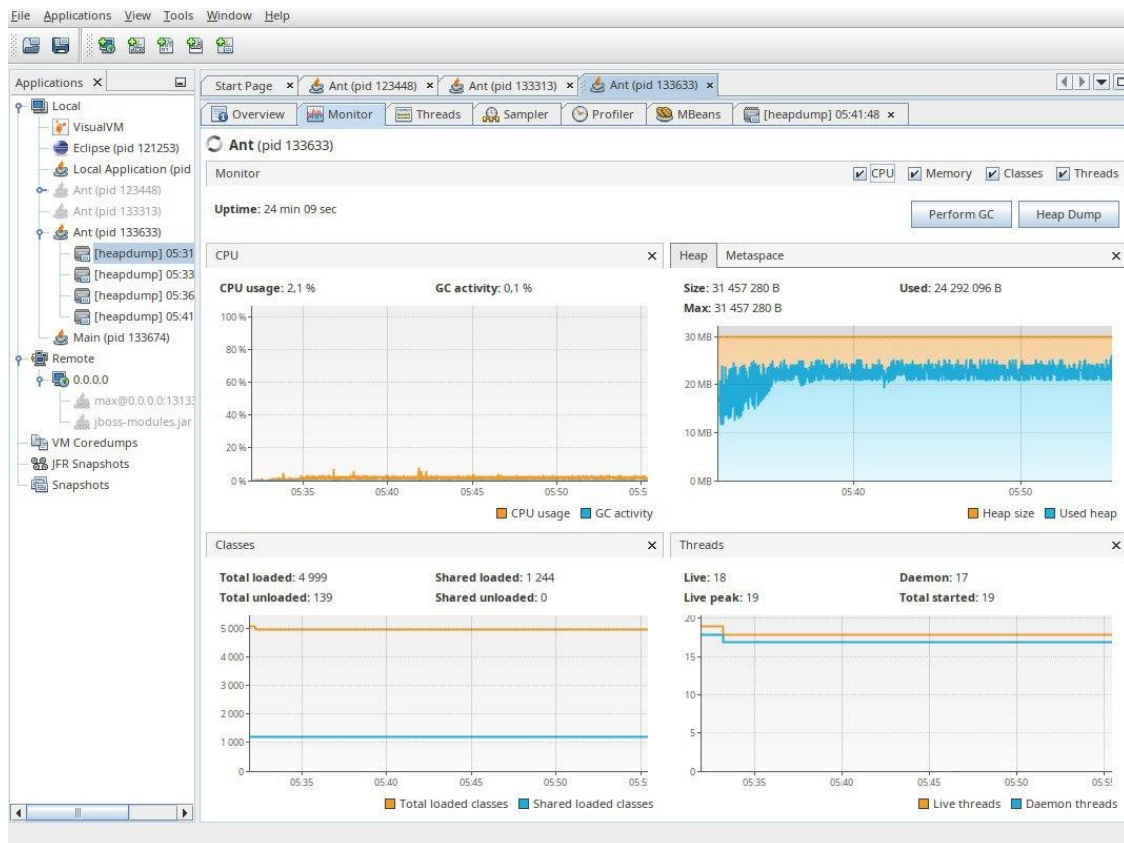
```
public void clearErrorMessages() {
    JavaScript.clearErrorMessages();
}
```

```
/**
 * Clears the accumulated script error messages.
 */
no usages
public static void clearScriptErrorMessages() {
    getScriptingEngine().clearErrorMessages();
}
```

Решением будет очистка списка с `_errorMessage` после выполнения очередного запроса.

```
while (true) {
    WebResponse response = sc.getResponse(request);
    System.out.println("Count: " + number++ + response);
    HttpUnitOptions.clearScriptErrorMessages();
}
```

Запустим программу. Теперь изменения памяти во времени менее пилообразные, GC работает более оптимально.



Программа работает стабильно и не падает с OutOfMemoryException.

```
[java] Count: 2005303[_response = com.meterware.servletunit.ServletUnitHttpResponse@46069bbd]
[java] Count: 2005304[_response = com.meterware.servletunit.ServletUnitHttpResponse@32dd0028]
[java] Count: 2005305[_response = com.meterware.servletunit.ServletUnitHttpResponse@75b8eb36]
[java] Count: 2005306[_response = com.meterware.servletunit.ServletUnitHttpResponse@64a8bc94]
[java] Count: 2005307[_response = com.meterware.servletunit.ServletUnitHttpResponse@4f95f8cb]
[java] Count: 2005308[_response = com.meterware.servletunit.ServletUnitHttpResponse@16de43aa]
[java] Count: 2005309[_response = com.meterware.servletunit.ServletUnitHttpResponse@3bd5b9b4]
[java] Count: 2005310[_response = com.meterware.servletunit.ServletUnitHttpResponse@1858b57]
[java] Count: 2005311[_response = com.meterware.servletunit.ServletUnitHttpResponse@676e7d4c]
[java] Count: 2005312[_response = com.meterware.servletunit.ServletUnitHttpResponse@75275ed6]
[java] Count: 2005313[_response = com.meterware.servletunit.ServletUnitHttpResponse@262f5cb1]
[java] Count: 2005314[_response = com.meterware.servletunit.ServletUnitHttpResponse@5b40a9c5]
[java] Count: 2005315[_response = com.meterware.servletunit.ServletUnitHttpResponse@11bd03de]
[java] Count: 2005316[_response = com.meterware.servletunit.ServletUnitHttpResponse@7958beba]
[java] Count: 2005317[_response = com.meterware.servletunit.ServletUnitHttpResponse@4f01dffc]
[java] Count: 2005318[_response = com.meterware.servletunit.ServletUnitHttpResponse@28f0ac3e]
[java] Count: 2005319[_response = com.meterware.servletunit.ServletUnitHttpResponse@3ed2ae5f]
[java] Count: 2005320[_response = com.meterware.servletunit.ServletUnitHttpResponse@398504c8]
[java] Count: 2005321[_response = com.meterware.servletunit.ServletUnitHttpResponse@191d03aa]
[java] Count: 2005322[_response = com.meterware.servletunit.ServletUnitHttpResponse@c9c3b58]
[java] Count: 2005323[_response = com.meterware.servletunit.ServletUnitHttpResponse@788895bf]
[java] Count: 2005324[_response = com.meterware.servletunit.ServletUnitHttpResponse@4360d585]
[java] Count: 2005325[_response = com.meterware.servletunit.ServletUnitHttpResponse@7f1a46e8]
[java] Count: 2005326[_response = com.meterware.servletunit.ServletUnitHttpResponse@61700b14]
[java] Count: 2005327[_response = com.meterware.servletunit.ServletUnitHttpResponse@1167d8]
[java] Count: 2005328[_response = com.meterware.servletunit.ServletUnitHttpResponse@35ba37dd]
[java] Count: 2005329[_response = com.meterware.servletunit.ServletUnitHttpResponse@4c817ff8]
[java] Count: 2005330[_response = com.meterware.servletunit.ServletUnitHttpResponse@1cd7ed48]
[java] Count: 2005331[_response = com.meterware.servletunit.ServletUnitHttpResponse@9fa89e6]
[java] Count: 2005332[_response = com.meterware.servletunit.ServletUnitHttpResponse@5b5d2574]
[java] Count: 2005333[_response = com.meterware.servletunit.ServletUnitHttpResponse@33043228]
[java] Count: 2005334[_response = com.meterware.servletunit.ServletUnitHttpResponse@27d2a548]
[java] Count: 2005335[_response = com.meterware.servletunit.ServletUnitHttpResponse@1344b5f5]
[java] Count: 2005336[_response = com.meterware.servletunit.ServletUnitHttpResponse@3d23abd6]
[java] Count: 2005337[_response = com.meterware.servletunit.ServletUnitHttpResponse@1526f7de]
[java] Count: 2005338[_response = com.meterware.servletunit.ServletUnitHttpResponse@74c32dc]
[java] Count: 2005339[_response = com.meterware.servletunit.ServletUnitHttpResponse@263aa202]
[java] Count: 2005340[_response = com.meterware.servletunit.ServletUnitHttpResponse@40c4416b]
```

6. Вывод

Во время выполнения работы я научился писать MBeans, изучил утилиты для мониторинга и профилирования JConsole и VisualVM, а также был научился по полученным данным определять утечки памяти и устранять их.